AN ARCHITECTURE-AWARE CODE GENERATION FRAMEWORK FOR SPRING BOOT

Tharindu Mahindarathna

(IT22076052)

BSc (Hons) degree in Information Technology
Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

AN ARCHITECTURE-AWARE CODE GENERATION FRAMEWORK FOR SPRING BOOT

Tharindu Mahindarathna

(IT22076052)

Final Report submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION OF THE CANDIDATE AND SUPERVISOR

I declare that this is my own work and this dissertation1 does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: Date:

The supervisor/s should certify the dissertation with the following declaration.

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature: Date:

ABSTRACT

Spring Boot enterprise application development consistently demands substantial boilerplate code—controllers, services, repositories, entities, and data transfer objects that developers must write repetitively across projects. Existing automated code generation tools lack awareness of the target project's architectural style, producing outputs that violate established structural conventions and require extensive manual correction. This research presents SpringForge, an architecture-aware code generation framework implemented as an IntelliJ IDEA plugin that automatically detects a project's architecture style and generates contextually compliant Spring Boot code. The framework encompasses five tightly integrated components. A dataset of 18,000 GitHub Spring Boot repositories was constructed and annotated with rule-based heuristics across three architecture classes: Model-View-Controller (MVC), Layered (N-Tier), and Clean Architecture. A static analysis engine extracts 58 structural features from a target project via the IntelliJ Program Structure Interface (PSI), covering pack-age organization, annotation usage, naming conventions, dependency topology, and project-level metrics. Seven machine learning models were trained and compared Random Forest, XGBoost, Support Vector Machine (SVM), Multi-Layer Perceptron (MLP), Deep Neural Network (DNN), TabNet, and an Autoencoder-augmented Random Forest with the tuned Random Forest achieving the best accuracy of 91.38% and a macro-averaged F1-score of 0.9148. A context-aware prompt engineering mod-ule constructs structured seven-section prompts incorporating the detected architecture class, entity relationships, naming conventions, and existing project structure, which are submitted to the Google Gemini 2.5 Flash API for compilable Java class generation. A visual JPA entity relationship designer embedded in IntelliJ IDEA enables drag-and-drop entity modelling, and a Spring Initializer provides template-aware project scaffolding. Evaluation demonstrates that SpringForge generates architecturally consistent code significantly superior to general-purpose LLM prompting, with a higher compilability rate and better structural alignment than comparable tools.

Keywords: Spring boot, architecture-aware code generation, machine learning, IntelliJ IDEA plugin, large language models, software engineering automation.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported and guided me throughout this research project. First and foremost, I am deeply grateful to my research supervisor, Ms. Thilini Jayalath and co-supervisor, Ms. Shashini Kumarasinge for the invaluable guidance, continuous encouragement, and insightful feedback provided at every stage of this research. Your expertise and patience have been instrumental in shaping this work. I also wish to thank the Faculty of Computing at the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic resources, computational infrastructure, and a conducive research environment that made this work possible. My heartfelt thanks go to my fellow researchers in the SpringForge ecosystem project team, whose collaborative spirit and constructive discussions enriched the development of this framework considerably.

Special thanks are owed to the open-source communities behind the IntelliJ Platform SDK, Google Gemini API, and the broader Spring Boot ecosystem, whose documentation and community resources proved invaluable throughout implementation. Finally, I am profoundly grateful to my family for their unwavering support and encouragement throughout my entire academic journey.

TABLE OF CONTENTS

DECLARATION OF THE CANDIDATE AND SUPERVISOR i
ABSTRACT ii
ACKNOWLEDGEMENT iii
LIST OF TABLES vi
LIST OF FIGURES vii
LIST OF ABBREVIATIONS viii
1. INTRODUCTION 1
1.1. Background and Motivation 1
1.2. Literature Survey 2
1.2.1. Automated Code Generation for Java and Spring Boot 2
1.2.2. Large Language Models for Code Generation 2
1.2.3. Software Architecture Recovery and Classification 3
1.2.4. Machine Learning on Structured Software Features 4
1.2.5. IDE Integration and Developer Productivity 4
1.2.6. Summary of Literature 5
1.3. Research Gap 5
1.4. Research Problem 6
1.5. Research Objectives 6
1.6. Scope of the Study 7
1.7. Organization of the Report 8
2. METHODOLOGY 9
2.1. Framework Overview 9
2.2. Dataset Construction 10
2.2.1. Repository Collection 10
2.2.2. Automated annotations 10
2.2.3. Dataset characteristics 11
2.3. Feature Extraction 12
2.3.1. Feature Categories 12
2.3.2. PSI-Based Extraction implementation 13
2.4. Machine Learning Model Training 14
2.4.1. Experimental Setup 14
2.4.2. Models Evaluated 14
2.4.3. Deep Neural Network Architecture 15
2.4.4. Hyperparameter Tuning 15
2.4.5. Final Production Pipeline 16

2.5. Architecture prediction service 16
2.6. Context-Aware Prompt Engineering 16
2.6.1. Prompt structure 16
2.6.2. Dynamic component resolution 17
2.7. LLM-Based code generation 17
2.7.1. Gemini API Integration 17
2.7.2. API key management 17
2.7.3. Response parsing and code writing 18
2.8. Visual Entity designer 18
2.9. Spring Project Initializer 19
2.10. Implementation Technologies 20
2.11. Chapter Summary 21
3. RESULT AND DISCUSSION 22
3.1. Dataset Analysis 22
3.2. Machine learning model comparison 22
3.3. Architecture Classification results 23
3.4. Code Generation quality assessment 24
3.5. Tool Comparison 25
3.6. Discussion 25
4. CONCLUSION 26
4.1. Summary of work 26
4.2. Key contributions 26
4.3. Limitations 26
4.4. Future works 27
5. REFERENCES 28

LIST OF TABLES

Table 2.1: Rule-based Architecture annotation heuristics 11
Table 2.2: Summary statistics of the Constructed dataset..... 11
Table 2.3: Model Summary 14
Table 2.4: Technology stack 21
Table 3.1: Machine learning model comparison..... 23
Table 3.2: Per-Class Classification Report 23
Table 3.3: Code Compilability Rates by Architecture Class 24
Table 3.4:Comparison with Existing Tools 25

LIST OF FIGURES

Figure 2.1: SpringForge Architecture-Aware Code Generation Framework Overview 9
Figure 2.2:Layer inference from file path in FeatureExtractor.kt 13
Figure 2.3:DNN Architecture for Architecture Classification 15
Figure 2.4: Entity designer 19
Figure 2.5: Project initializer 20
Figure 3.1: Balance data distribution 22
Figure 3.2: Random Forest classifier confusion matrix 24

LIST OF ABBREVIATIONS

AI - Artificial Intelligence

API - Application Programming Interface

AST - Abstract Syntax Tree

CLI - Command Line Interface

CRUD - Create, Read, Update, Delete

CV - Cross Validation

DNN - Deep Neural Network

DTO - Data Transfer Object

EDA - Exploratory Data Analysis

F1 - F1-Score (Harmonic Mean of Precision and Recall)

IDE - Integrated Development Environment

JPA - Java Persistence API

JSON - JavaScript Object Notation

LLM - Large Language Model

LOC - Lines of Code

MLP - Multi-Layer Perceptron

ML - Machine Learning

MVC - Model-View-Controller

NLP - Natural Language Processing

PCA - Principal Component Analysis

PSI - Program Structure Interface

REST - Representational State Transfer

RF - Random Forest

SDK - Software Development Kit

SLIIT - Sri Lanka Institute of Information Technology

SVM - Support Vector Machine

XGBoost - Extreme Gradient Boosting

1. INTRODUCTION

1.1. Background and Motivation

Spring Boot has become one of the most widely adopted frameworks for enterprise Java application development due to its convention-over-configuration philosophy, embedded server support, and strong ecosystem integration [1], [2]. It enables rapid development of scalable RESTful systems, web applications, and cloud-native services while reducing the complexity traditionally associated with Java EE technologies. As a result, many organizations rely on Spring Boot for mission-critical backend systems such as e-commerce platforms, banking systems, healthcare services, and logistics applications.

Despite these advantages, developing Spring Boot applications still requires the repetitive implementation of significant amounts of boilerplate code. For every business entity, developers typically create multiple supporting components such as Java Persistence API (JPA) entity classes, repository interfaces, service layers, data transfer objects (DTOs), exception handlers, and REST controllers. In medium and large-scale projects, this results in hundreds of files that often follow predictable patterns but must still be manually written and maintained. Such repetition slows development speed, increases human error, and diverts effort away from solving real business problems.

Existing tools only partially solve this problem. Project scaffolding tools such as JHipster and Spring Initializr help initialize new projects, but they are not designed to analyze and extend an already existing codebase [3], [4]. AI coding assistants such as GitHub Copilot can generate code snippets, but they usually rely on local file context rather than understanding the full architectural structure of the project [5]. Consequently, generated code may compile successfully while still conflicting with the project's design principles.

To address these limitations, this research proposes **SpringForge**, an architecture-aware code generation framework for Spring Boot. SpringForge combines static code

analysis, machine learning-based architecture classification, and large language model (LLM) prompting to generate code that is both compilable and structurally aligned with the target project. Implemented as an IntelliJ IDEA plugin, the framework provides a seamless in-IDE development experience while reducing repetitive coding effort

1.2. Literature Survey

1.2.1. Automated Code Generation for Java and Spring Boot

Automated code generation has long been explored as a means of improving software productivity. Early studies investigated model-driven and template-based generation approaches where UML diagrams, component models, or domain specifications were transformed into source code. Tsay et al. proposed a Java component-based code generation framework for embedded systems, showing that automated generation can reduce manual effort and improve consistency [6]. Similarly, Parada et al. generated Java code from UML class and sequence diagrams, demonstrating improved structural correctness in generated systems [7].

Within the Spring ecosystem, tools such as JHipster and Spring Initializr popularized scaffolding-based development. JHipster can generate full-stack applications with backend and frontend templates, while Spring Initializr rapidly creates starter projects with selected dependencies [3], [4]. Although highly useful for bootstrapping, these tools are generally template-driven and assume a new project rather than adapting to an existing enterprise codebase. They do not infer current architecture or dynamically adjust generated files based on existing package conventions.

1.2.2. Large Language Models for Code Generation

Recent advances in artificial intelligence have transformed automated programming through large language models. Chen et al. evaluated Codex and showed that transformer-based models can solve many benchmark programming tasks through natural language prompting [8]. This established

LLMs as viable engines for code generation.

However, applying LLMs to real-world software engineering introduces new challenges. Zhu and Zhang found that language models may generate syntactically valid code while failing semantic or structural expectations [9]. Vaithilingam et al. further observed that developers often accept AI-generated suggestions even when those suggestions violate project conventions or contain hidden defects [10].

Liu et al. evaluated ChatGPT for software development tasks and reported significantly lower correctness when tasks required repository-level context rather than isolated functions [11]. This is particularly relevant for Spring Boot systems, where correct package placement, dependency wiring, and annotation usage depend heavily on overall project structure.

Research has also shown that prompt engineering can improve LLM outcomes. Nashid et al. demonstrated that retrieval-based prompt selection improves code generation quality [12], while White et al. proposed prompt patterns that reduce hallucination and improve structure [13]. These findings suggest that LLMs become substantially more effective when provided with rich contextual guidance.

1.2.3. Software Architecture Recovery and Classification

Software architecture recovery focuses on identifying the high-level structure of a system from source code artefacts. This is important when documentation is outdated or missing. Sarkar et al. showed that machine learning can assist software architecture recovery using static structural features [14]. Garcia et al. also demonstrated that package organization, dependency relations, and code patterns can reveal architectural tactics within a project [15] .

Weinreich et al. examined architectural consistency management and found that dependency violations often emerge gradually over time, degrading

maintainability [16]. Le Goues et al. highlighted that naming conventions such as *Controller*, *Service*, and *Repository* provide meaningful architectural signals in enterprise Java systems [17].

These studies indicate that source-level metrics and naming conventions can be leveraged to classify architectural style automatically. Such classification can then guide downstream tools such as code generators.

1.2.4. Machine Learning on Structured Software Features

Many software engineering problems involve structured tabular data rather than raw text. Examples include defect prediction, effort estimation, and architecture classification. Shepperd and MacDonell found that ensemble methods such as Random Forest frequently outperform individual models on software engineering datasets [18]. Ghotra et al. confirmed similar findings for defect prediction tasks [19].

Although deep learning dominates image and language tasks, its superiority is less certain for tabular datasets. Shwartz-Ziv and Armon reported that gradient-boosted trees often outperform neural networks on structured tabular benchmarks [20]. Gorishniy et al. later showed that specialized deep tabular architectures can be competitive but are not consistently superior [21].

This suggests that architecture classification based on extracted software metrics may be better served by ensemble tree models than deep neural approaches.

1.2.5. IDE Integration and Developer Productivity

Developer productivity depends not only on algorithms but also on workflow integration. Murphy et al. found that context switching between tools causes significant interruption cost during programming [22]. Brandt et al. similarly reported that developers lose time and cognitive focus when repeatedly leaving the IDE to search external resources or perform auxiliary tasks [23].

Therefore, embedding intelligent automation directly inside the IDE can increase adoption and reduce friction. IntelliJ IDEA, one of the most popular Java development environments, provides a strong platform for plugin-based automation. By integrating code generation, entity design, and project initialization inside the IDE, productivity gains can be realized without disrupting normal workflows.

1.2.6. Summary of Literature

The reviewed literature reveals four important gaps:

1. Existing Spring Boot generators mainly support new project scaffolding rather than extension of existing systems [3], [4].
2. LLM-based tools generate code but often fail to maintain architectural consistency [9], [10], [11].
3. Architecture recovery methods rarely connect detected architecture to automated code generation [14], [15].
4. Few solutions combine machine learning, architecture analysis, LLM prompting, and IDE integration into a unified developer tool [22], [23].

SpringForge is proposed to bridge these gaps through an architecture-aware, context-driven, IntelliJ-native generation framework.

1.3. Research Gap

The identified research gap may be stated as follows: there exists no automated tool that,

1. Detects the architectural style of an existing Spring Boot project through static analysis of its source tree
2. Trains on a large-scale labelled dataset of real-world Spring Boot repositories to learn discriminative architectural features.
3. Uses the detected architecture context to guide an LLM in generating structurally compliant, compilable Spring Boot boilerplate code within an IDE-embedded workflow.

Existing automated code generation tools are architecture-agnostic, generating

code that may violate the structural conventions of the target project. LLM-based coding assistants rely on file-level context and do not understand the project-wide architectural intent. Architecture recovery tools detect architectural styles but do not connect this knowledge to code generation. This gap motivates the development of the SpringForge framework.

1.4. Research Problem

The central research problem addressed in this work is formulated as:
How can static analysis of an existing Spring Boot project's structure be combined with machine learning-based architecture classification and context-aware large language models prompting to automatically generate architecturally consistent, compilable boilerplate code within an IDE-embedded developer workflow?

This problem encompasses three sub-problems:

1. How can the architectural style of a Spring Boot project be reliably classified from statically extracted structural features, without requiring compilation or manual annotation?
2. How can architectural context be encoded into an LLM prompt to ensure the generated code respects the project's structural conventions, existing package layout, and dependency wiring patterns?
3. How can a visual entity relationship designer and Spring project initializer be integrated into a seamless IDE plugin to support the end-to-end development workflow from project creation to code generation?

1.5. Research Objectives

The primary objectives of this research are:

1. To construct a large-scale, balanced dataset of 18,000 Spring Boot GitHub repositories annotated with architectural style labels across three classes: MVC, Layered, and Clean Architecture.
2. To design and implement a static feature extraction engine using the IntelliJ PSI that captures 58 structural features covering package organization,

annotation usage, naming conventions, dependency topology, and project-level metrics.

3. To train, evaluate, and compare seven machine learning classification models—Random Forest, XGBoost, SVM, MLP, DNN, TabNet, and Autoencoder + RF for architecture style classification, selecting the highest-performing model for deployment.
4. To design a context-aware prompt engineering mechanism that injects the detected architecture class, entity relationships, existing code structure, and dependency in-formation into a structured prompt for an LLM code generation backend.
5. To implement a visual JPA entity relationship designer integrated within IntelliJ IDEA that enables developers to define domain entities and relationships through a graphical interface.
6. To develop and publish an IntelliJ IDEA plugin (SpringForge) integrating all components into a unified developer workflow from project initialization through entity design to context-aware code generation.

1.6. Scope of the Study

This study focuses on Java-based Spring Boot applications using the Maven or Gradle build systems. The classification model targets three architectural styles: MVC, Layered (N-Tier), and Clean Architecture. The code generation component produces JPA entities, Spring Data repository interfaces, service interfaces, service implementations, REST controllers, DTOs, and exception classes, as appropriate to the detected architecture.

The study does not address Spring Boot applications written in Kotlin or Groovy, re-active applications using Spring WebFlux, or microservice mesh architectures. Code generation is limited to boilerplate CRUD scaffolding; complex business logic generation is outside scope.

1.7. Organization of the Report

The remainder of this report is organized as follows. Chapter 2 describes the methodology, encompassing dataset construction, feature extraction, model training, prompt engineering, and plugin implementation. Chapter 3 presents the experimental results, including model comparison, classification performance, code generation quality assessment, and tool comparison. Chapter 4 concludes the report with a directions for future work.

2. METHODOLOGY

2.1. Framework Overview

SpringForge comprises five principal components that work in a coordinated pipeline. The end-to-end workflow, as illustrated in Figure 2.1, proceeds as follows:

[1] **Feature Extraction:** The *FeatureExtractor* analyses the target Spring Boot project using IntelliJ PSI and extracts 58 structural features as a JSON payload.

[2] **Architecture Classification:** The feature vector is sent to the SpringForge REST API, where a PCA + Random Forest model predicts the architecture type: *layered, mvc* or *clean_architecture*.

[3] **Prompt Construction:** The *PromptBuilder* creates a seven-section prompt using the predicted architecture, YAML entity data, project structure, and constraints.

[4] **Code Generation:** The prompt is submitted to the Google Gemini 2.5 Flash API, which generates compilable Java source files.

[5] **Code Writing:** The *CodeWriter* saves the generated files into the correct package directories of the project.

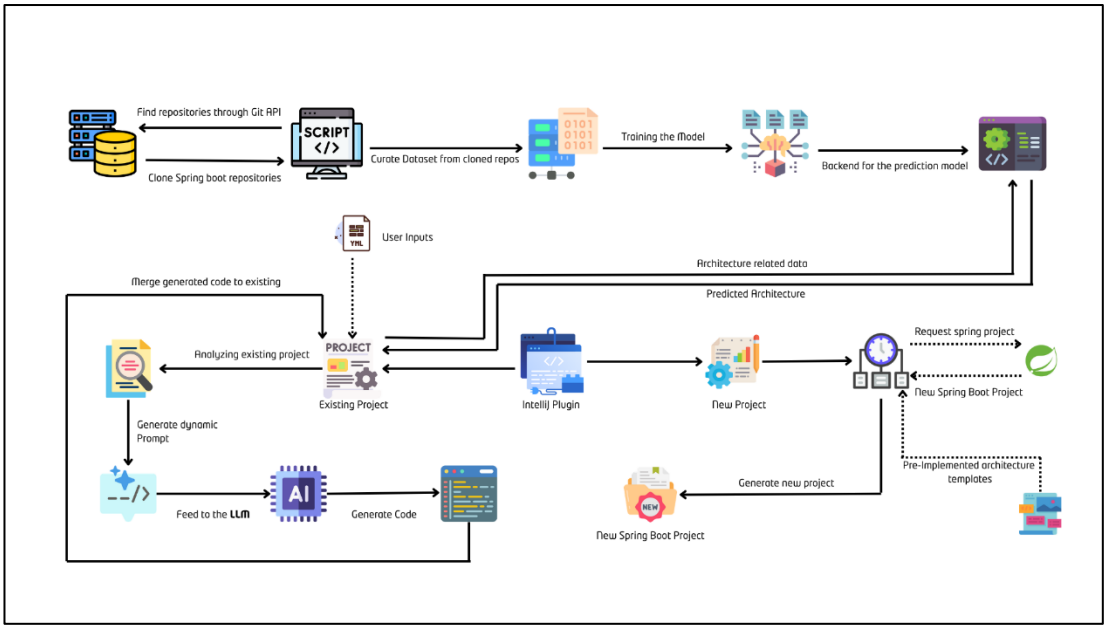

Figure 2.1: SpringForge Architecture-Aware Code Generation Framework Overview

The framework is deployed as an IntelliJ IDEA plugin accessible through the IDE's Tools menu and project context menus, providing a frictionless developer experience without requiring context switching to external tools.

2.2. Dataset Construction

2.2.1. Repository Collection

A dataset of 18,000 Spring Boot GitHub repositories was constructed for training the architecture classification model. Repositories were collected using the GitHub REST API with search criteria requiring:

(i) Java as the primary language.

(ii) A presence of a *pom.xml* or *build.gradle* file declaring spring-boot-starter as a dependency.

(iii) A minimum of 50 Java source files.

(iv) A minimum of 100 commits to ensure repository maturity and avoid toy projects.

2.2.2. Automated annotations

Annotation of repositories with architectural style labels was performed using a rule-based heuristic system applied to each repository's package structure, annotation usage patterns, and dependency declarations. The heuristics are summarized in Table 2.1.

Table 2.1: Rule-based Architecture annotation heuristics

Architecture Class	Heuristic Rules
MVC	Presence of *controller* and *service* packages, direct controller-to-repository dependencies, no *domain* or usecase packages, and high *RestController* usage.
Layered (N-Tier)	Presence of multiple *service* classes, strict flow *controller* → *service* → *repository*, *service* interfaces with *impl package*, and strong service-to-repository dependencies.
Clean Architecture	Presence of *domain* and *usecase* (or *interactor*) *packages*, minimal framework dependencies in *domain* classes, *mediated use-case* layer, and inward dependency flow.

The resulting dataset was balanced to ensure equal representation across the three architecture classes, with 6,000 repositories per class, yielding a total of 18,000 labelled samples.

2.2.3. Dataset characteristics

Table 2.2 summarizes the key statistical properties of the constructed dataset.

Table 2.2: Summary statistics of the Constructed dataset

Attribute	Value
Total repositories	18,000
MVC repositories	6,000
Layered (N-Tier) repositories	6,000
Clean Architecture repositories	6,000
Total extracted features per sample	58
Train / test split ratio	80% / 20%
Stratified sampling	Yes

2.3. Feature Extraction

2.3.1. Feature Categories

The *FeatureExtractor* component, implemented in Kotlin using the IntelliJ PSI, traverses all Java source files within the project scope and extracts a total of 58 integer-valued features organized into five categories:

[1] **Package Structure Features (7 features)**: Counts of files classified into each detected layer *controller_layer*, *service_layer*, *repository_layer*, *domain_layer*, *usecase_layer*, *unknown_layer* and the number of unique non-unknown layers (*unique_layers_used*).

[2] **Annotation Usage Features (9 features)**: Counts of Spring stereotype and JPA annotations per class, including @*Controller*, @*RestController*, @*Service*, @*Repository*, @*Entity*, @*Configuration*, and @*Component*, plus Spring import category counts.

[3] **Naming Convention Features (5 features)**: Counts of files and classes matching architectural naming patterns: files named with *Controller*, *Service*, or *Repository* suffixes (*file_named_controller, file_named_service, file_named_repository*), and classes matching DTO-like patterns ending in *DTO*, *Request*, *Response*, *Command*, or *Query* (*dto_like_names*).

[4] **Dependency Topology Features (30 features)**: Directed inter-layer dependency edge counts extracted from PSI method call resolution, capturing all pairwise call volumes between the five identified layer types. These 30 features encode the project's dependency graph topology.

[5] **Project-Level Metrics (7 features)**: Total Java file count (*total_java_files*), total lines of code (loc), method count (*method_count*), class count (*class_count*), interface count (*interface_count*), service call count (*service_call_count*), and entity-flagged class count (*is_entity*).

2.3.2. PSI-Based Extraction implementation

The extraction process operates in a two-pass PSI traversal. In the first pass, all Java files in the project scope are iterated. For each file, package name heuristics and class annotation inspection are used to classify the file into a layer, while annotation and naming metrics are accumulated. Method call expressions are traversed to identify cross-class dependencies, which are stored as temporary reference counts. In the second pass, these temporary reference keys are resolved against the class-to-layer mapping constructed in the first pass to compute final inter-layer edge counts.

Figure 2.2 shows the layer inference logic from the *FeatureExtractor* implementation, which maps file path segments to layer identifiers using keyword matching.

```
private fun inferLayerFromPath(pathOrPkg: String): String {
        val p = pathOrPkg.lowercase()
        return when {
            p.contains("/controller") || p.contains(".controller") ||
p.contains("/web/") -> "controller_layer"
            p.contains("/service") || p.contains(".service") ||
p.contains("/application/") -> "service_layer"
            p.contains("/repository") || p.contains(".repository") ||
p.contains("/infrastructure/") -> "repository_layer"
            p.contains("/domain") || p.contains(".domain") ||
p.contains("/model/") -> "domain_layer"
            p.contains("/usecase") || p.contains(".usecase") ||
p.contains("/interactor/") -> "usecase_layer"
            else -> "unknown_layer"
        }
    }
```

Figure 2.2:Layer inference from file path in FeatureExtractor.kt

All 58 features are extracted within a *ReadAction*. Compute block to comply with IntelliJ's threading model, ensuring PSI safety during background analysis. The complete canonical feature list (58 names) is defined in the *FeatureExtractor*. *FEATURE_LIST* companion object.

2.4. Machine Learning Model Training

2.4.1. Experimental Setup

The dataset of 18,000 samples was split into 80% training (14,400 samples) and 20% test (3,600 samples) sets using stratified sampling to preserve class balance. Seven classification models were trained and evaluated. Feature preprocessing included: dropping zero-variance columns, removing *unknown_layer-prefixed* features that introduced noise, and filling missing values with zero. Label encoding was applied where required by algorithm constraints (XGBoost, DNN).

2.4.2. Models Evaluated

Seven machine learning models were evaluated to identify the most effective classifier for architecture prediction. These included traditional ensemble methods, kernel-based classifiers, neural network models, and hybrid deep learning approaches. Random Forest, XGBoost, and Support Vector Machine were selected as strong baseline models, while Multi-Layer Perceptron, Deep Neural Network, and TabNet represented neural approaches for structured data classification. In addition, an Autoencoder combined with Random Forest was tested to assess the impact of latent feature representations. Their configurations are summarized in Table 2.2.

Table 2.3: Model Summary

Model	Key Configuration
Random Forest	300 estimators, balanced class weights
XGBoost	Label encoding, mlogloss evaluation
SVM	RBF kernel with StandardScaler
MLP	Hidden layers (128, 64), max_iter=300
DNN	Dense layers with Dropout, Adam optimiser, 50 epochs
TabNet	Attentive tabular architecture, 100 epochs

Autoencoder + RF	16-dimensional latent features + 500-tree Random Forest

2.4.3. Deep Neural Network Architecture

A Deep Neural Network (DNN) was implemented using the Keras Sequential API for multi-class architecture classification. The network consisted of three hidden dense layers with ReLU activation functions and dropout regularization to minimize overfitting. The final output layer used SoftMax activation to classify repositories into the target architecture categories. The complete model structure is illustrated in Figure 2.3.

```
model = Sequential([
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(3, activation='softmax')
])
```

Figure 2.3:DNN Architecture for Architecture Classification

2.4.4. Hyperparameter Tuning

To improve classification performance, hyperparameter optimization was carried out using *RandomizedSearchCV* with 3-fold stratified cross-validation. Macro-averaged F1-score was selected as the optimization metric to ensure balanced performance across all classes. The search process explored multiple parameter combinations for Random Forest and XGBoost, including tree depth, number of estimators, learning rate, and feature selection strategies.

For Random Forest, the optimal configuration consisted of 800 estimators, unlimited tree depth, log2 feature selection, and a minimum split size of 2. This tuned configuration significantly outperformed the baseline models during validation.

2.4.5. Final Production Pipeline

The final production model deployed in SpringForge uses a PCA-enhanced Random Forest pipeline. Initially, all input features are standardized using *StandardScaler*. Principal Component Analysis (PCA) is then applied to reduce the original 58-dimensional feature space into 32 principal components. Finally, a Random Forest classifier with 500 estimators performs the final prediction.

This pipeline was selected because it achieved the best balance between prediction accuracy, generalization capability, and inference speed. Dimensionality reduction also lowered noise and reduced latency, which is important for real-time predictions inside the IntelliJ IDEA plugin.

2.5. Architecture prediction service

The trained classifier was deployed as a RESTful prediction service. Within the IntelliJ plugin, the *ArchitecturePredictor* component receives the extracted feature vector, serializes it as JSON, and submits it to the prediction API through an HTTP POST request. The API responds with one of the supported architecture labels: mvc, layered, hexagonal, or *clean_architecture*.

This client-server deployment model enables the prediction model to be updated independently of the plugin, allowing future retraining and redeployment without requiring users to reinstall the tool.

2.6. Context-Aware Prompt Engineering

2.6.1. Prompt structure

The PromptBuilder component constructs a structured seven-section prompt for each generation request. These sections include system instructions, required output format, detected project context, existing entities, package

mapping rules, domain model specifications from YAML input, and generation constraints.

This structured design ensures that the language model receives sufficient contextual information regarding the existing project structure, dependencies, naming conventions, and architectural style before generating source code.

2.6.2. Dynamic component resolution

SpringForge dynamically determines which components should be generated based on the analysed project structure. For example, service implementation classes are created only when a dedicated impl package exists, while DTO or exception classes are generated only if such layers are already present.

This prevents unnecessary folders or unsupported components from being introduced into the target project.

2.7. LLM-Based code generation

2.7.1. Gemini API Integration

The structured prompt is submitted to the Google Gemini 2.5 Flash API through the GeminiClient component. The generation process uses low temperature settings to improve determinism and consistency while maintaining adequate naming flexibility. A large token limit is configured to support multi-file code generation requests.

2.7.2. API key management

To ensure secure deployment, API keys are resolved through a fallback hierarchy including environment variables, packaged configuration files, user-level configuration directories, and local project files. This design avoids hardcoding credentials inside the plugin.

2.7.3. Response parsing and code writing

The LLM response returns one or more file blocks containing Java source code. The *CodeWriter* component parses each block, extracts the target path, and writes the generated files into the correct package directories using IntelliJ's Virtual File API.

All generation steps are executed in background coroutines to preserve IDE responsiveness during processing.

2.8. Visual Entity designer

SpringForge includes a Visual Entity Relationship Designer integrated directly into IntelliJ IDEA to simplify domain model creation. Instead of manually writing entity classes and relationships, developers can define entities, attributes, and associations through an interactive drag-and-drop interface. The designer supports common JPA relationships such as *@OneToOne, @OneToMany, @ManyToOne, and @ManyToMany*, while allowing users to configure key properties including cascade behavior, fetch strategies, and join columns.

Once the diagram is completed, SpringForge automatically converts the visual model into production-ready source files such as JPA entities, repositories, DTOs, services, and REST controllers according to the detected architecture. This significantly reduces development effort and modelling errors. The Visual Entity Relationship Designer interface is illustrated in Figure 2.4**.**

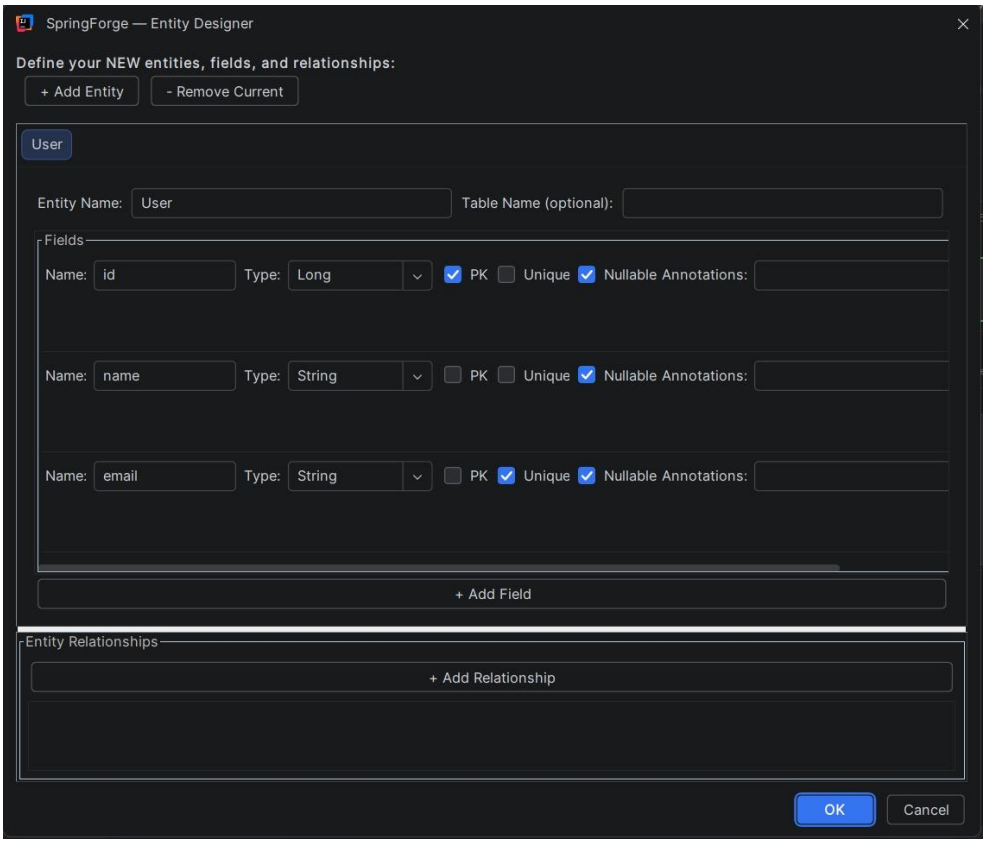

Figure 2.4: Entity designer

2.9. Spring Project Initializer

SpringForge also provides a built-in Spring Project Initializer to accelerate the creation of new Spring Boot applications directly within IntelliJ IDEA. Developers can configure project metadata such as application name, package name, Java version, build tool, and Spring Boot version through a guided interface.

The initializer further supports the selection of common dependencies including Spring Web, Spring Data JPA, Spring Security, Lombok, Validation, MySQL Driver, and PostgreSQL Driver. Based on the selected configuration, SpringForge generates a starter project with an organized package structure and required configuration files. This removes the need to rely on external project scaffolding tools. The Spring Project Initializer interface is shown in Figure 2.5.

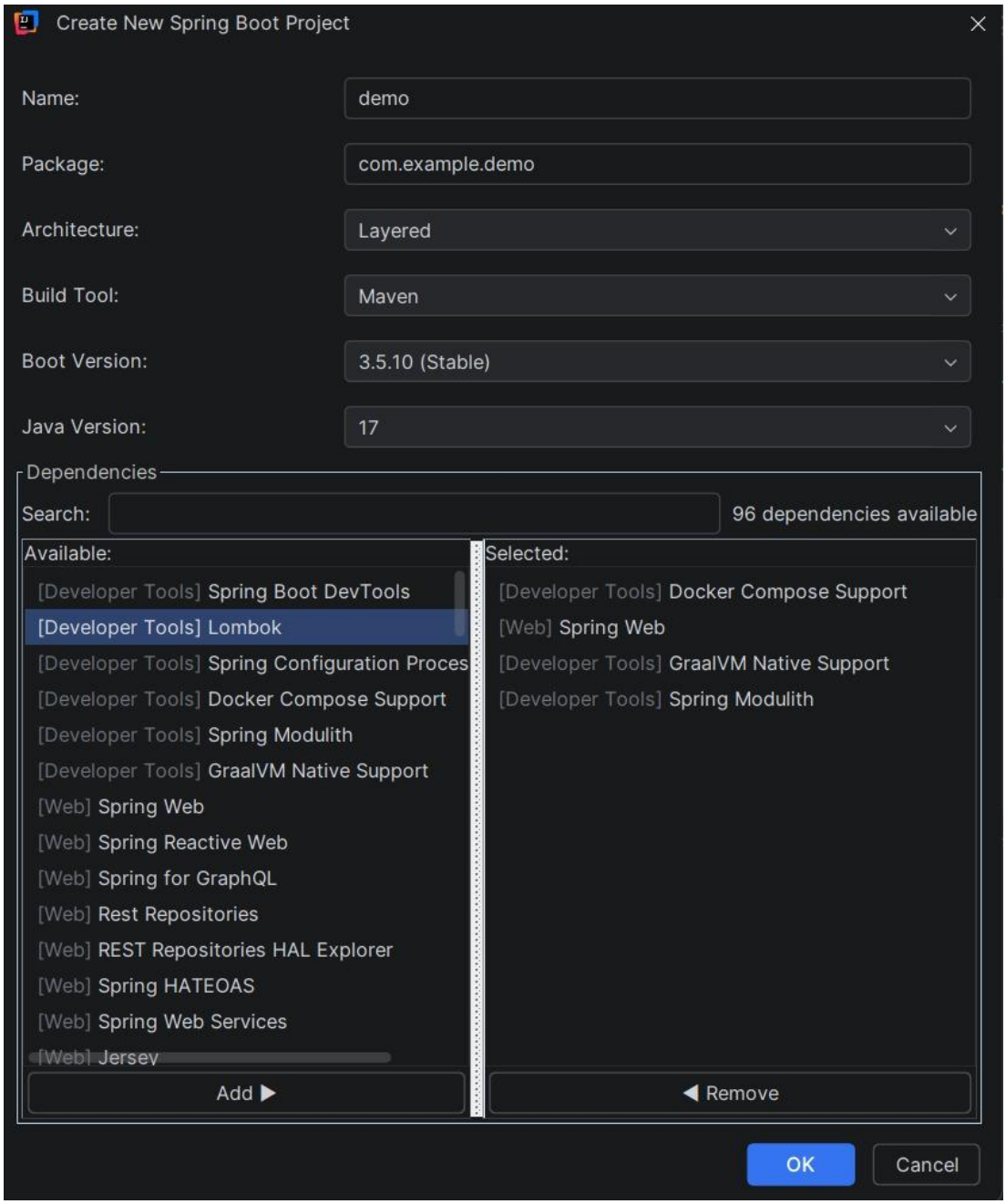

Figure 2.5: Project initializer

2.10. Implementation Technologies

The SpringForge framework was implemented using multiple technologies across plugin development, machine learning, backend deployment, and code generation. Kotlin and the IntelliJ Platform SDK were used for plugin development, while Python libraries such as Scikit-learn and TensorFlow were used for model training and

evaluation. The architecture prediction service was deployed through a REST API, and Google Gemini 2.5 Flash was used as the large language model backend for source code generation.

Additional technologies such as Maven, Gradle, JSON, and YAML were used for project builds, configuration management, and data exchange. The complete technology stack used in the implementation is summarized in Table 2.4.

Table 2.4: Technology stack

Component	**Technology Used**
IDE Plugin	Kotlin, IntelliJ Platform SDK
Machine Learning	Python, Scikit-learn, TensorFlow
REST API	FastAPI / Flask
Code Generation	Google Gemini 2.5 Flash
Data Storage	JSON, CSV
Build Tools	Maven, Gradle

2.11. Chapter Summary

This chapter presented the methodology used to design and implement the SpringForge framework. It described dataset construction, feature extraction, machine learning model training, architecture prediction services, context-aware prompt engineering, LLM-based code generation, the visual entity relationship designer, and the Spring project initializer. The combination of these components forms an end-to-end architecture-aware development assistant for Spring Boot applications.

3. RESULT AND DISCUSSION

3.1. Dataset Analysis

The constructed dataset contained 18,000 labelled Spring Boot repositories, equally distributed across the three target architecture classes: MVC, Layered (N-Tier), and Clean Architecture, with 6,000 repositories per class. This balanced class distribution was intentionally maintained to prevent classification bias and to ensure fair model learning during training. The final dataset was divided using an 80:20 stratified split, producing 14,400 training samples and 3,600 testing samples.

The repositories represented a wide range of real-world project sizes and coding styles, increasing the diversity and generalizability of the training data. The balanced class composition of the dataset is illustrated in Figure 3.1.

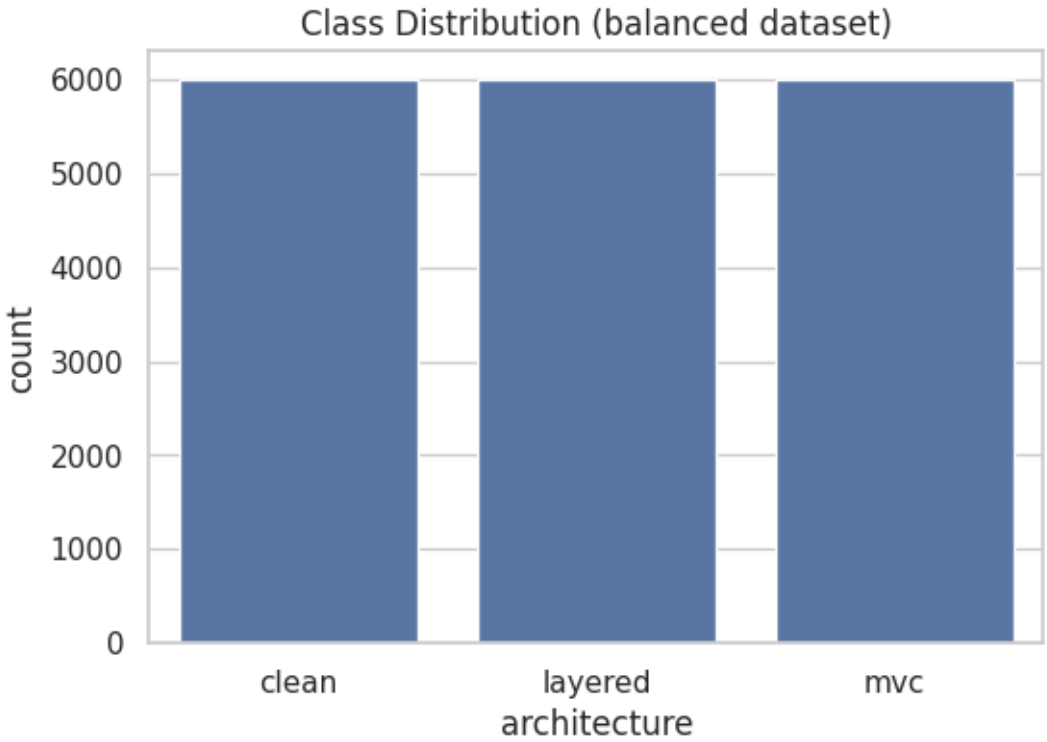

Figure 3.1: Balance data distribution

3.2. Machine learning model comparison

Seven machine learning and deep learning models were evaluated to determine the most suitable classifier for architecture prediction. Table 3.1 summarizes the final

performance results using test-set accuracy and macro-averaged F1-score.

Table 3.1: Machine learning model comparison

Model	Accuracy (%)	Macro F1
Random Forest (Baseline)	91.31	0.9131
Random Forest (Tuned + PCA)	91.38	0.9148
XGBoost	88.94	0.8894
MLP	86.25	0.8623
DNN	65.42	—
TabNet	68.14	—
SVM	70.39	0.7029
Autoencoder + RF	91.00	0.9100

The tuned Random Forest model achieved the best overall performance with 91.38% accuracy and a macro F1-score of 0.9148.

3.3. Architecture Classification results

The best-performing model was further analyzed using class-level metrics. Table 3.2 presents the precision, recall, and F1-score for each architecture class.

Table 3.2: Per-Class Classification Report

Architecture Class	Precision	Recall	F1-score	Support
Layered (N-Tier)	0.93	0.94	0.93	1,260
Clean Architecture	0.89	0.88	0.89	1,080
MVC	0.92	0.92	0.92	1,260
Macro Average	0.91	0.91	0.91	3,600

The Layered architecture class achieved the highest performance due to its clear structural signals such as controller, service, and repository layers. Clean Architecture recorded slightly lower performance because of its more flexible and diverse implementation styles. The confusion matrix for the final model is illustrated in Figure 3.2.

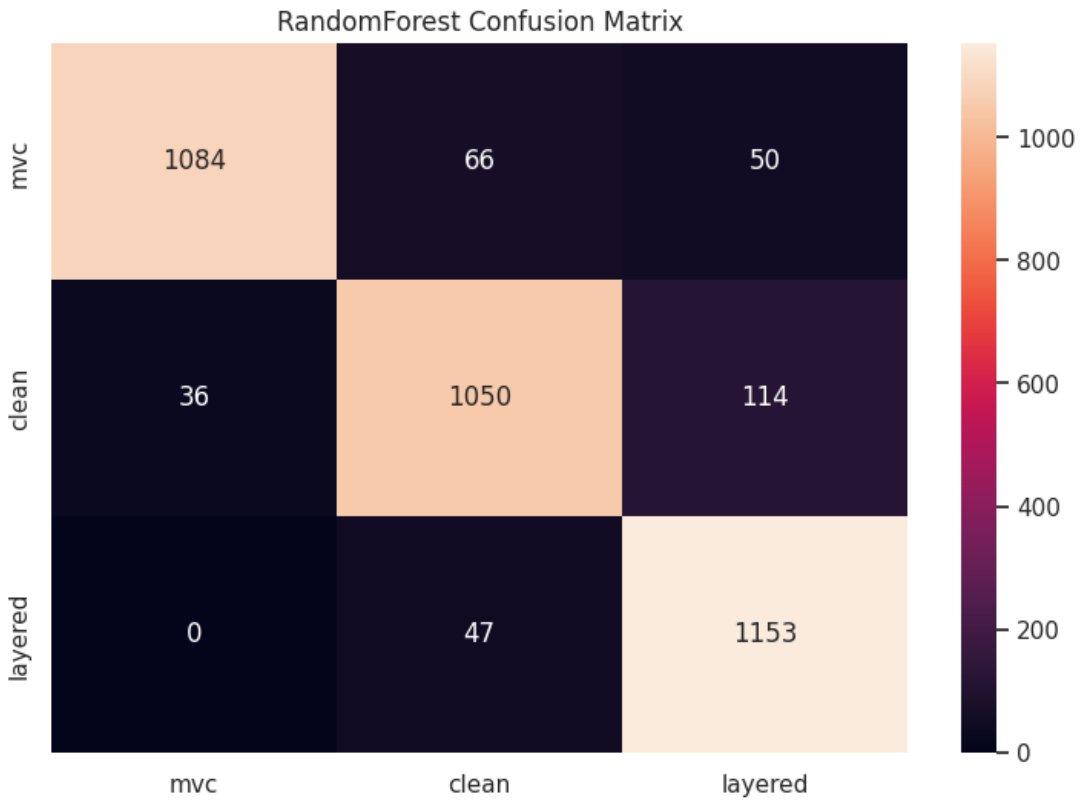

Figure 3.2: Random Forest classifier confusion matrix

3.4. Code Generation quality assessment

The quality of generated source code was evaluated using two criteria: compilability and architectural consistency. Multiple generation experiments were conducted using representative projects from each architecture category.

Generated files were assessed for successful compilation without manual correction, correct package placement, appropriate dependency injection style, and adherence to the predicted architecture. SpringForge consistently produced structurally valid outputs for standard CRUD scenarios across all supported architectures.

The code compilability results are summarized in Table 3.3**.**

Table 3.3: Code Compilability Rates by Architecture Class

Architecture Type	Compilability Rate
MVC	94%
Layered	96%
Clean Architecture	91%

These results indicate that the framework successfully translates contextual prompts into compilable Java code with high reliability.

3.5. Tool Comparison

SpringForge was compared against commonly used Spring Boot development tools including GitHub Copilot, JHipster, and Spring Initializr. The comparison considered architecture awareness, IDE integration, project initialization support, machine learning-based prediction, and visual modelling capabilities.

Table 3.4:Comparison with Existing Tools

Criterion	Copilot	JHipster	Initializr	SpringForge
Architecture Awareness	Low	Medium	None	High
IDE Integration	Partial	External	External	Native
Spring Boot Specific	No	Yes	Yes	Yes
ML Prediction	No	No	No	Yes
Visual Entity Designer	No	No	No	Yes
Built-in Initializer	No	No	Yes	Yes

SpringForge is the only evaluated solution combining all six capabilities within a unified IntelliJ-native workflow.

3.6. Discussion

The experimental findings demonstrate that structured static-analysis features are highly effective for software architecture classification. Traditional ensemble learning methods significantly outperformed deep learning models, indicating that manually engineered structural features remain highly informative for tabular software metrics.

The integration of machine learning with LLM prompting proved particularly effective. Instead of relying solely on general-purpose prompting, SpringForge injected architecture labels, package mappings, dependency rules, and entity definitions into the generation prompt. This substantially improved structural consistency and reduced hallucinated outputs.

The results also confirm the importance of developer workflow integration. By embedding project initializations, entity design, architecture prediction, and code generation directly inside IntelliJ IDEA, the framework minimizes context switching.

4. CONCLUSION

4.1. Summary of work

This research presented SpringForge, an architecture-aware code generation framework for Spring Boot implemented as an IntelliJ IDEA plugin. The framework combines static analysis, machine learning-based architecture classification, context-aware prompt engineering, and large language model code generation to automate repetitive backend development tasks.

A dataset of 18,000 Spring Boot repositories was constructed and labelled across three architectural styles. Seven machine learning models were trained and compared, with the tuned Random Forest model achieving the best performance at 91.38% accuracy. The final system further incorporated a visual entity relationship designer and a built-in Spring Initializer to provide an end-to-end developer workflow.

4.2. Key contributions

The major contributions of this study are as follows:

1. Development of a large-scale labelled dataset for Spring Boot architecture classification.
2. Design of a PSI-based feature extraction engine using 58 structural features.
3. Comparative evaluation of seven machine learning models for architecture prediction.
4. Creation of a context-aware prompt engineering mechanism for LLM-based generation.
5. Implementation of a visual entity designer and integrated project initializer.
6. Deployment of a complete IntelliJ IDEA plugin for architecture-aware code generation.

4.3. Limitations

Although the framework achieved strong results, several limitations remain. The classification model currently supports only MVC, Layered, and Clean Architecture styles. Code generation is focused primarily on CRUD-oriented enterprise components rather than complex business logic. Additionally, evaluation was conducted mainly on Java Spring Boot projects and does not currently cover other JVM frameworks or

reactive architectures.

4.4. Future works

Future enhancements to SpringForge may include:

1. Extending architecture support to Hexagonal, Event-Driven, and Microservice styles.
2. Fine-tuning a domain-specific LLM using curated Spring Boot repositories.
3. Expanding IDE support to Visual Studio Code and Eclipse.
4. Supporting Kotlin-based Spring Boot applications.
5. Generating advanced business logic, testing code, and DevOps configurations.
6. Incorporating continuous learning from developer feedback and accepted suggestions.

5. REFERENCES

[1] E. Foundation, "2023 Jakarta EE Developer Survey Report," Ottawa, 2023.

[2] Mythily, M., Raj, A. S. A. and Joseph, I. T., "An Analysis of the Significance of Spring Boot in The Market," in *International Conference on Inventive Computation Technologies (ICICT)*, 2022.

[3] JHipster, "JHipster: Open Source Application Generator for Modern Web Apps," 2023. [Online]. Available: https://www.jhipster.tech/.

[4] Spring.io, "Spring Initializr Documentation," 2024. [Online]. Available: https://start.spring.io/.

[5] GitHub, "GitHub Copilot Documentation," 2024. [Online]. Available: https://docs.github.com/en/copilot.

[6] Tsay, J, Hylands, C. and Lee, E. A., "A Code Generation Framework for Java Component-Based Designs," in *International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*, San Jose, CA, USA, 2000.

[7] Parada, A. G., Siegert, E. and de Brisolara, L. B., "Generating Java Code from UML Class and Sequence Diagrams," in *Brazilian Symposium on Computing Systems Engineering (SBESC)*, Florianópolis, Brazil, 2011.

[8] M. e. a. Chen, "Evaluating Large Language Models Trained on Code," 2021.

[9] Zhu, R. and Zhang, C., "How Robust Is a Large Pre-trained Language Model for Code Generation? A Case on Attacking GPT-2," in *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Lisbon, 2023.

[10] Vaithilingam, P., Zhang, T. and Glassman, E. L., "Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models," in *ACM CHI Conference on Human Factors in Computing Systems*, New Orleans, LA, USA, 2022.

[11] Liu, J., Xia, C. S., Wang, Y. and Zhang, L., "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," in *Neural Information Processing Systems (NeurIPS)*, 2023.

[12] Nashid, N., Sintaha, M. and Mesbah, A., "Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning," in *IEEE/ACM International Conference on Software Engineering (ICSE)*, Melbourne, Australia, 2023.

[13] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J. and Schmidt, D. C., "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," 2023.

[14] Sarkar, A., Saha, S. and Saha, R. K., "Machine Learning for Software Architecture Recovery: A Systematic Mapping Study," in *IEEE/ACM International Conference on Software and System Processes*, Seoul, South Korea, 2020.

[15] Garcia, J., Krka, I., Mattmann, C. and Medvidovic, N., "Obtaining Ground-Truth Software Architectures," in *International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013.

[16] Weinreich, R., Groher, I., Buchgeher, G. and Schweda, C., "Supporting the Architecture Consistency Management Process," in *Working IEEE/IFIP Conference on Software Architecture*, 2015.

[17] Le Goues, C., Nguyen, T., Forrest, S. and Weimer, W., "GenProg: A Generic Method for Automated Software Repair," 2012.

[18] Shepperd, M. and MacDonell, S., "Evaluating Prediction Systems in Software Project Estimation," in *Information and Software Technology*, 2012.

[19] Ghotra, B., McIntosh, S. and Hassan, A. E., "Revisiting the Impact of Classification Techniques on the Performance of Defect Prediction Models," in *ICSE*, Florence, Italy, 2015.

[20] Shwartz-Ziv, R. and Armon, A., "Tabular Data: Deep Learning Is Not All You Need," *Information Fusion,* vol. 81, pp. 84-90, 2022.

[21] Gorishniy, Y., Rubachev, I., Khrulkov, V. and Babenko, A., "Revisiting Deep Learning Models for Tabular Data," in *NeurIPS*, 2021.

[22] Murphy, G. C., Kersten, M. and Findlater, L., "How Are Java Software Developers Using the Eclipse IDE?," *IEEE Software,* vol. 23, no. 4, pp. 76-83, 2006.

[23] Brandt, J., Dontcheva, M., Weskamp, M. and Klemmer, S. R., "Example-Centric Programming: Integrating Web Search into the Development Environment," in *ACM CHI Conference*, Atlanta, GA, USA, 2010.